

Samah Keshiro
A03

Github repo: <https://github.com/samahkeshiro/SSW567/tree/main>

The screenshot shows the GitHub Actions interface for the 'test-workflow'. On the left, a sidebar lists workflows: 'test-workflow' (9s, 43m ago) and 'hw03a-tests' (7s). The main panel displays the 'test-workflow' details, showing a green checkmark for 'Success' and a search bar for jobs. A job named 'hw03a-tests' is highlighted with a green checkmark and a duration of 7s. The interface includes navigation links for 'Insights', 'Rerun', and a search bar for jobs.

The screenshot shows the GitHub repository page for 'ssw567' by 'samahkeshiro'. The page displays the 'README.md' file, which includes a 'PASSED' status and a description of the repository. The README content is as follows:

GitHubApi-hw03a

PASSED

Given a GitHub user ID, this program retrieves the list of repositories owned by that user, along with the number of commits in each repository, using GitHub's public REST API (no authentication needed).

Files

- `github_repo_stats.py` — the main program. Also runnable directly: `python3 github_repo_stats.py <github-username>`
- `test_github_repo_stats.py` — unit tests (pure-logic tests plus tests against the real GitHub API).
- `requirements.txt` — dependencies (requests).
- `DESIGN_NOTES.md` — write-up on the design/testing decisions made for this assignment.
- CI: tests run on CircleCI on every push, configured in `.circleci/config.yml` at the repository root.

Running it yourself

```
PS C:\Users\katey\OneDrive - stevens.edu\SSW 567\A03> python3 -m unittest -v
test_counts_multiple_commits (test_github_repo_stats.TestCountCommits.test_counts_multiple_commits) ... ok
test_single_commit (test_github_repo_stats.TestCountCommits.test_single_commit) ... ok
test_zero_commits_for_empty_list (test_github_repo_stats.TestCountCommits.test_zero_commits_for_empty_list) ... ok
test_empty_list_for_user_with_no_repos (test_github_repo_stats.TestExtractRepoNames.test_empty_list_for_user_with_no_repos) ... ok
test_extract_names_from_multiple_repos (test_github_repo_stats.TestExtractRepoNames.test_extract_names_from_multiple_repos) ... ok
test_single_repo (test_github_repo_stats.TestExtractRepoNames.test_single_repo) ... ok
test_matches_assignment_required_format (test_github_repo_stats.TestFormatRepoCommitLine.test_matches_assignment_required_format) ... ok
test_zero_commits_still_formats_correctly (test_github_repo_stats.TestFormatRepoCommitLine.test_zero_commits_still_formats_correctly) ... ok
test_every_result_is_name_and_non_negative_count (test_github_repo_stats.TestRealGitHubApi.test_every_result_is_name_and_non_negative_count) ... ok
test_known_repo_has_commits (test_github_repo_stats.TestRealGitHubApi.test_known_repo_has_commits) ... ok
test_known_repo_is_listed (test_github_repo_stats.TestRealGitHubApi.test_known_repo_is_listed) ... ok
test_nonexistent_repo_raises_http_error (test_github_repo_stats.TestRealGitHubApi.test_nonexistent_repo_raises_http_error) ... ok
test_nonexistent_user_raises_http_error (test_github_repo_stats.TestRealGitHubApi.test_nonexistent_user_raises_http_error) ... ok
test_single_repo_lookup_matches_full_listing (test_github_repo_stats.TestRealGitHubApi.test_single_repo_lookup_matches_full_listing) ... ok

Ran 14 tests in 9.372s

OK
PS C:\Users\katey\OneDrive - stevens.edu\SSW 567\A03>
```

Reflection:
Designing for testability

I split the code into functions that call the GitHub API and functions that only process data that's already been fetched. The data-processing functions are tested with small hand-made lists. The functions that call GitHub are tested against the real API, using richkempinski's public ``hellogitworld`` repo.

Testing against a live API

Real data changes, so my tests check things that should stay true (the repo exists and has more than 0 commits) instead of exact commit counts. GitHub also limits unauthenticated requests to 60 per hour, so the test class fetches the user's data once and reuses it.

Edge cases I found by thinking like a tester

- **Pagination:** GitHub returns only 30 items per page by default. Simply counting the list would undercount any repo with more than 30 commits, so my code requests pages of 100 until it reaches the last page. I couldn't reliably test this with real public data, since it needs a repo with over 100 commits that won't change.
- **Empty repositories:** For a repo with no commits, GitHub returns a 409 error instead of an empty list. My code treats this as 0 commits, but this isn't tested yet for the same reason.
- **Nonexistent users and repos:** These return a 404, which raises an ``HTTPError``. I test both against the real API.

Challenges and future improvements

The biggest challenge was GitHub's rate limit. While testing, I hit GitHub's limit of 60 unauthenticated requests per hour and got a 403 error, so all of my API tests failed even though my code hadn't changed. I had to check GitHub's rate-limit page and wait for the limit to reset before I could test again. This showed me that tests depending on a live service can fail for reasons that have nothing to do with the code.

Live data was another challenge. Commit counts change over time, so I couldn't assert exact numbers and had to check facts that should stay true instead. Some edge cases, like empty repos and pagination, also couldn't be tested reliably with real data. Faking the API's responses would solve both problems, which is what I'll do next with mocking.